Predicting the Movement of AMZN stock price

Souhoon Lee

August 6, 2025

https://colab.research.google.com/drive/1o7s6WGShaE0TBACTZOgSfn4enwQjevrk?usp=sharing

https://www.kaggle.com/datasets/tanavbajaj/yahoo-finance-all-stocks-dataset-daily-update

Introduction

Quantitative finance is a very lucrative field that many have told me to consider trying. I wanted to try to take this final project and use it as an opportunity to explore my old (and flawed) ideas of what quantitative traders actually build. I used to believe that quant finance purely looked at price trends and built models that recognized patterns within the charts and then traded options and stocks based on predictions that they made from said patterns, but after spending more time researching I realized that this was not quant finance but perhaps a more nuanced version of algorithmic day trading.

Nonetheless, I wanted to see how that old view would fare in predicting the volatility of a stock's price. I chose Amazon, as it is a very well-established company and because recent earnings reports and other news have caused Amazon stock price (and my own personal portfolio) to fall. I went onto Kaggle to find market data for Amazon stock price, and I found this data set. It contained historical stock prices for a little under 600 different companies, Amazon included. While I could have simply just taken the AMZN ticker from the dataset, I wanted to also experiment with comparisons to the market, as each stock trades in a very interlinked environment. Specifically, I used the total market data to calculate the alpha of AMZN every day. Alpha is a financial metric that is used to determine whether or not a stock is outperforming or underperforming relative to some benchmark, which in my case was the entire market.

The calculation for the alpha of a stock is simply portfolio return minus benchmark return. I calculated portfolio return as simply the closed price minus the open price $(P_C - P_O)$ and I derived benchmark return by finding the percent change in the total market $(P_{C,market} - P_{O,market})/P_{O,\,market}$ and multiplying that by the open price of AMZN. Using those two return values, I was able to calculate the alpha of a stock. Furthermore, I then took the accumulated alpha (though I am not sure if this is a standard or fallacious practice) to track the general trends of alpha over time and determine whether or not Amazon was outperforming the market.

Code

My first issue was finding the total market data: while I had data for around 600 individual companies, they were not compiled in any way, and there were too many files for Google Colab to handle. I was then forced to turn to VSCode (the .py file will be attached to the assignment) in order to create the total market data.

```python
import os
import pandas as pd
import chardet
import datetime

csv_folder = "/Users/souhoonlee/Desktop/YISS ML/Final assignment/datasets/Data_market"
csv_files = [f for f in os.listdir(csv_folder) if f.endswith('.csv')]

start_date = "2000-01-03 00:00:00+00:00"
end_date = "2023-01-02 00:00:00+00:00"
columns_to_sum = ["Date","Open", "High", "Low", "Close"]

paths = [os.path.join(csv_folder, f) for f in csv_files]

sum_df = pd.read_csv(paths[0],usecols=columns_to_sum)

sum_df['Date'] = pd.to_datetime(sum_df['Date'], utc = True, format='%Y-%m-%d %H:%M:%S%z')
sum_df["Date"] = sum_df["Date"].dt.normalize()

sum_df = sum_df[(sum_df['Date'] >= start_date) & (sum_df['Date'] <= end_date)]
sum_df.set_index('Date', inplace=True)

for path in paths[1:]:
    df = pd.read_csv(path,usecols=columns_to_sum)
    df['Date'] = pd.to_datetime(df['Date'], utc=True, format='%Y-%m-%d %H:%M:%S%z')
    df["Date"] = df["Date"].dt.normalize()
    df = df[(df['Date'] >= start_date) & (df['Date'] <= end_date)]
    df.set_index('Date', inplace=True)
    sum_df = sum_df.add(df, fill_value=0)

sum_df.to_csv("TOTAL_MARKET.csv", index=True)
```

I downloaded all of the files to a folder local to my desktop, and then opened up the folder through VSCode. I selected which columns I wanted to combine in the columns_to_use

variable, and I created start_date and end_date variables because I needed to make sure that every data point was aligned perfectly together in time. I then transformed the Date column to a DateTime object, and after slicing each dataframe to contain only the elements within my desired time frame, I set the date column to be the index column to avoid any errors in summing up all of the dataframes. Once I had compiled everything, I sent it out as a csv file.

```
import requests
import chardet
import io
import pandas as pd

#loading the data
total_market_csv = "https://drive.google.com/file/d/18schLXuQxpNJvf1bdKBp0chuHM96l2dc/view?usp=sharing"
TMID = total_market_csv.split("/")[-2]
TMCSV = f"https://drive.google.com/uc?id={TMID}&export=download"

total_market = requests.get(TMCSV)
market_csv = total_market.content

mrk_enc = chardet.detect(market_csv)

MARKET = pd.read_csv(io.StringIO(market_csv.decode(mrk_enc['encoding'])))

AMZN_csv = "https://drive.google.com/file/d/1-1fffeTJX3a7mQLcU9E38osYxnJr9iMn/view?usp=sharing"
AID = AMZN_csv.split("/")[-2]
ACSV = f"https://drive.google.com/uc?id={AID}&export=download"

amzn_read = requests.get(ACSV)
AMZN_csv = amzn_read.content

AMZN_enc = chardet.detect(AMZN_csv)

AMZN = pd.read_csv(io.StringIO(AMZN_csv.decode(AMZN_enc['encoding'])))
```

```
[191] #slicing the data to match our total market time frame.
      AMZN = AMZN.drop(columns = ["Dividends","Stock Splits","Volume"])
      AMZN["Date"] = pd.to_datetime(AMZN["Date"],utc =True,format = "%Y-%m-%d %H:%M:%S%z")
      AMZN["Date"] = AMZN["Date"].dt.normalize()

      start_date = "2000-01-03 00:00:00+00:00"
      end_date = "2023-01-02 00:00:00+00:00"
      AMZN = AMZN[(AMZN['Date'] >= start_date) & (AMZN['Date'] <= end_date)]

      AMZN.set_index("Date", inplace = True)
```

The first two cells were simply loading and reading the data into google colab, along with taking only the desired time slice from the Amazon dataframe. Doing so, I was able to create a market dataframe that was uniform in structure with the Amazon dataframe.

```
[192] #Lets find change in price (close - open)
      AMZN["Change_percentage"] = ((AMZN["Close"] - AMZN["Open"])/AMZN["Open"])
      AMZN.head()

      MARKET["Change_percentage"] = ((MARKET["Close"] - MARKET["Open"])/MARKET["Open"])
      #MARKET.head()
```

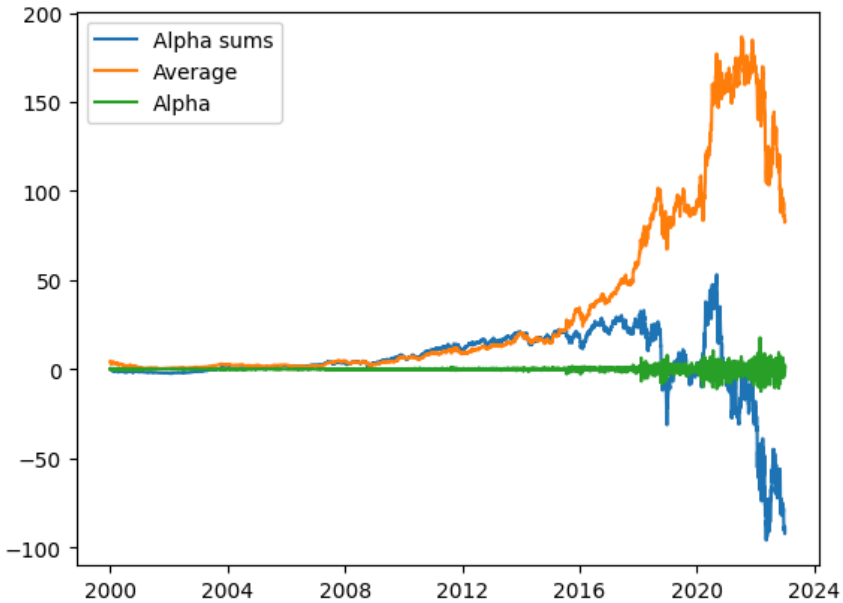
```
alpha_vals = (AMZN["Close"] - AMZN["Open"])
for i in range(len(alpha_vals)):
  alpha_vals[i] += AMZN["Open"][i]*MARKET["Change_percentage"][i]

AMZN["Alpha"] = alpha_vals

AMZN["Alpha sums"] = AMZN["Alpha"].cumsum()

AMZN["Average"] = (AMZN['Open'] + AMZN["Close"])/2

AMZN = AMZN.drop(columns = ["Open","High","Low","Close"])

AMZN.head()
```

The data in the dataframes alone were not enough, however. I wanted to find the change in percentage for both dataframes (although later down the line I realized that the change_percentage column for my Amazon dataframe was unnecessary). Once I had the change_percentage column for my market dataframe, I was able to create both the alpha and the subsequent alpha sums columns for Amazon, which then concluded my feature selection.

I did not clean up my data set (aside from taking a subset dataframe for structure purposes), as I would only be considering the Amazon stock price and change in stock price. I saw no need to scale my data, and I had checked previously for NA values (although the code is not visible, while trying to merge the csv files to make the market dataset, I came across a lot of errors and ran into a lot of bugs, and the process for fixing these issues lead me to finding out the absence of NA values in my datasets).

My main goal was to see if it was possible to predict the movement of Amazon's stock price based on the change in summed alpha, as I saw the summed alpha data values as a strong representation of all the other features within the Amazon dataframe as well as a very good comparison to the market, and to account for volatility of the market I chose to use a polynomial fit regression model (a slight branch off of the linear regression model).

However, the volatility of the market also leads to old data simply becoming noise. If we look at the graph on the right, we can see that the price spikes up drastically (Average is the average price for a day) and also falls drastically, meaning that from



around 2016 onwards, any data point before 2015 will be an outlier. Thus, I decided to use a form of Time Series Cross Validation in order to determine what “window” would be the best to use as a model in predicting the movement of the stock price within the next 10 days.

```
#Using summed alpha, I want to predict the average cost over the next 10 days.
#Doing so, I will use the polynomial fit for a higher power regression model.

import random
import numpy as np
from sklearn.metrics import mean_squared_error
import warnings

warnings.filterwarnings("ignore", message="Polyfit may be poorly conditioned")
#There is a rankwarning for the polynomial fit, but as I only saw this fit as a viable option for the time series regression, I chose to ignore it.

random.seed(42)

def time_regression(data,n,p): #n is number of samples in training, p is point selected for starting.
#Function will return a list:[output, error]
  train = data.iloc[p-n:p].copy()
  test = data.iloc[p:p+10].copy()
  n = 1
  error = None
  while error == None or error > 5 or n <= 15:
    coef = np.polyfit(train["Alpha sums"],train["Average"],n)
    func = np.poly1d(coef)
    train_target = func(test["Alpha sums"])
    error = mean_squared_error(train_target,test["Average"])
    if n == 15 or error < 5:
      break
    else:
      n += 1
  return [train_target,error]

def time_cv(data,list_of_samples,iterations):
  length = data.shape[0]
  error = {}
  points = random.sample(range(max(list_of_samples)+10,length-10),iterations)
  for n in list_of_samples:
    error[n] = []
    for point in points:
      error[n].append(time_regression(data,n,point)[1])
  for err in error:
    error[err] = np.mean(error[err])
  min_error = list_of_samples[0]
  for sample in list_of_samples[1:]:
    if error[sample] < error[min_error]:
      min_error = sample
  return error[min_error],min_error
```

I was unable to figure out a way to incorporate a scikit learn function into my specific hyperparameter tuning, so I ended up creating the functions from scratch. First, the model was time_regression, where I simply tried to find some polynomial function with a power less than 15 that would fit some specified window n at some specified point p: the model would then predict the average of the 10 values after point p, and return both this prediction as well as the error, calculated by MSE.

To determine the best window n, I created a function time_csv that simply took a list of samples (a list of possible windows), and a number of points to test on (iterations). It would then compile the list of errors from all points and use it in a hash table where the key was the

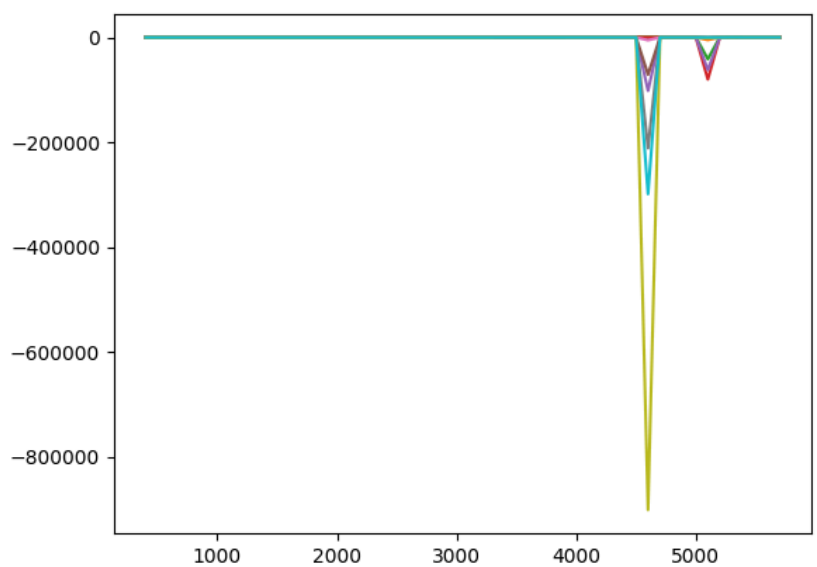
associated window. I then found the average of each window's error list and returned the lowest average error with the window associated.

The window associated with the lowest average error then became my optimal value n, and I used it as the best window parameter for my model. The reason I didn't do a train/test split was because randomly choosing a training and testing set from my dataframe would mess up the time-order, so in order to test the data I applied my model with optimal n on "every" point in the data set (in reality, every 100 points just so that I would not have to wait for hours for the graph to finish).

```
[209] training_k = [1,2,3,4,5,20,30,50,100,200,300,400]
      m = 1000

      out = time_cv(AMZN,training_k,m)
      out
      #this will take ~ 1 minute to run.
```

```
(np.float64(9913122710.969261), 400)
```

After using the model on 1000 different points and with the training window options in the training_k list, I found the optimal window for my model to be 400. After testing this data on all points on the data set, I found the following error graph. It seems that, around the time that amazon prices spike, the error in our model's predictive power increases (decreases?) dramatically.



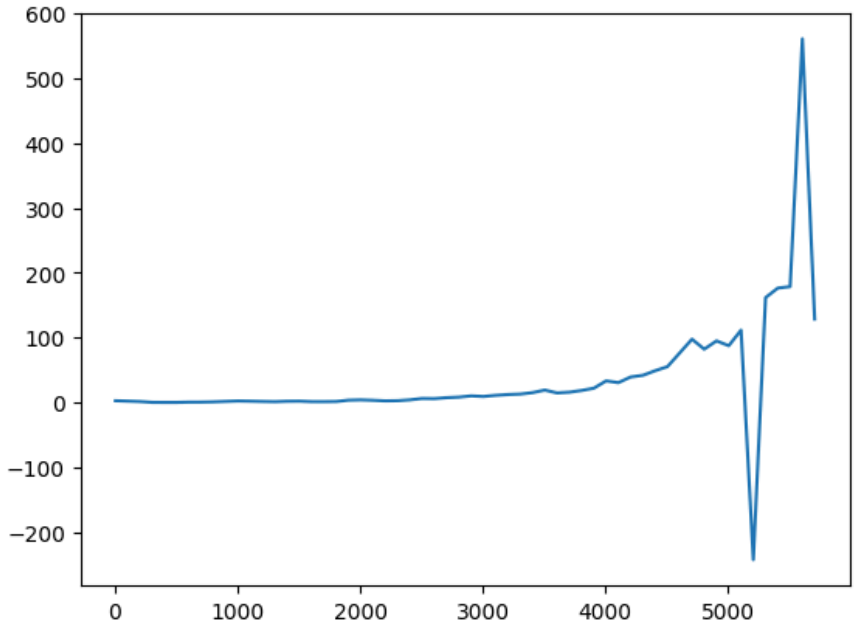
I was surprised at this conclusion: I figured that the volatility of the market would lead to smaller windows being more accurate (~1 day), not 400 days. While the graph itself didn't surprise me, as historical data cannot predict future events such as the pandemic, I was curious as to why the error was so large and also why the optimal window was equally large.

```
warnings.filterwarnings("ignore", message="Polyfit may be poorly conditioned")
#There is a rankwarning for the polynomial fit, but as I only saw this fit as a viable option for the time series regression, I chose to ignore it.

random.seed(42)

def time_regression(data,n,p): #n is number of samples in training, p is point selected for starting.
#Function will return a list:[output, error]
  train = data.iloc[p-n:p].copy()
  test = data.iloc[p:p+10].copy()
  n = 1
  error = None
  while error == None or error > 5 or n <= 15:
    coef = np.polyfit(train["Alpha sums"],train["Average"],n)
    func = np.poly1d(coef)
    train_target = func(test["Alpha sums"].mean())
    error = abs(train_target - test["Average"].mean())
    if n == 15 or error < 5:
      break
    else:
      n += 1
  return [train_target,error]

def time_cv(data,list_of_samples,iterations):
  length = data.shape[0]
  error = {}
  points = random.sample(range(max(list_of_samples)+10,length-10),iterations)
  for n in list_of_samples:
    error[n] = []
    for point in points:
      error[n].append(time_regression(data,n,point)[1])
  for err in error:
    error[err] = np.mean(error[err])
  min_error = list_of_samples[0]
  for sample in list_of_samples[1:]:
    if error[sample] < error[min_error]:
      min_error = sample
  return error[min_error],min_error
```

I then slightly altered the code: I thought that perhaps, instead of using mean squared error, what if I simply find the distance between the predicted value our model gives us from the input of the average of all "Alpha sums" values in our test set and the actual average of the test "average" set.

```
training_k = [1,2,3,4,5,20,30,50,100,200,300,400,1000]
m = 1000

out = time_cv(AMZN,training_k,m)
out

(np.float64(55.792922869663975), 4)
```

This small edit seemed to do the trick (in much faster time as well), and we can see that our error graph is a little bit more realistic, as we see larger deviations from 0 error as time moves closer to Amazon's peak in stock price.



Conclusion

It seems that using pure stock price compared to market performance is not enough to predict change in a stock's value. Although the final result did align with what I believed, that a smaller window for analysis can lead to more accurate predictions, this project seemed to prove once again that the market is unpredictable.

Reflection

I really enjoyed this course. I did come from a background of (a little) knowledge in Machine learning and a little bit of coding, but I found all of the lectures extremely helpful and insightful. I learned a lot of the inner workings and logic behind machine learning models like DBSCAN, KNN, K-Means, and even began to understand artificial neural networks. I was a little disappointed that there wasn't more coding, but I do understand that as an intro course, asking for a lot of coding is mean. I did enjoy how all of the projects gave me the opportunity to build the models or build other functions to help support the premade models. I think that I gained a lot of valuable coding experience from this course. I used ChatGPT to debug a little

(although, I had to go to stackoverflow and API documentation more often than not due to limitations in the free version of ChatGPT). I took this course because I was interested in machine learning and AI engineering, and (after this final project especially), I plan to start recruiting for Quant finance later in my second year of university and (if I can't get an offer), I hope to learn more about AI and machine learning and work developing modern LLMs (and also, I hope to finish the small LLM project that I started).

Advice to Next cohort

Go through all the notes and listen in class: the notes were very thorough, and Professor Lim did an amazing job at explaining all of the concepts that we covered. Also, don't be afraid to challenge yourself and do more than the assignment asks for!